Übungsblatt 06: IP, Subnetting und Routing mit Mininet

Rechnernetze und Netzwerkprogrammierung, WS 2026/27

Raphael Bleier, Devis, Fabio, Gruppe 1

Erklärung

Gemessen am 30.10.2026 mit Mininet 2.3.0 und Python 3.13.3. `xterm` ist hier nicht installiert, deshalb liefen `ip` und `ping` in den Netzwerk-Namespaces der Knoten. Das ist dieselbe Sicht wie in einem Terminal auf dem Knoten. Aufgaben 6.1 bis 6.3 verwenden die Vorlage vor der Änderung aus 6.7. Nichts wurde auf Moodle hochgeladen.

Erklärung

Die Vorlage setzt die Adressen nicht in `topology.py`, sondern in `start_lab.py`. Deshalb stehen in 6.1 bis 6.3 die Werte aus `configure_base_network`, nicht geratene Adressen. Die Bridge aus 6.7 ist in diesen Messungen noch nicht drin.

Aufgabe 6.1

Interfaces und IP-Adressen von client, r1, r2 und server mit `ip -br addr` und `ip addr` untersuchen. Tabelle mit Hostname, Interface, IP-Adresse, Präfix und Netzwerk. Welche Interfaces liegen im selben IP-Netz, welche sind durch Router getrennt?

Die Netzadresse ist Adresse UND Maske. Bei /24 fallen die letzten 8 Bit weg, bei /30 die letzten 2.

Hostname	Interface	IP-Adresse	Präfix	Netzwerk
client	lo	127.0.0.1	/8	127.0.0.0/8
client	client-eth0	10.0.1.2	/24	10.0.1.0/24
r1	lo	127.0.0.1	/8	127.0.0.0/8
r1	r1-eth0	10.0.1.1	/24	10.0.1.0/24
r1	r1-eth1	10.0.12.1	/30	10.0.12.0/30
r2	lo	127.0.0.1	/8	127.0.0.0/8
r2	r2-eth0	10.0.12.2	/30	10.0.12.0/30
r2	r2-eth1	10.0.2.1	/24	10.0.2.0/24
server	lo	127.0.0.1	/8	127.0.0.0/8
server	server-eth0	10.0.2.2	/24	10.0.2.0/24

`ip -br addr` auf dem Client zeigte `client-eth0` mit `10.0.1.2/24` und `lo` mit `127.0.0.1/8`. Bei r1 standen `r1-eth0` mit `10.0.1.1/24` und `r1-eth1` mit `10.0.12.1/30`. Bei r2 standen `r2-eth0` mit `10.0.12.2/30` und `r2-eth1` mit `10.0.2.1/24`. Der Server hatte `server-eth0` mit `10.0.2.2/24`. Alle sechs Dateninterfaces waren `UP`.

Gleiches IP-Netz, direkte Nachbarn, kein Router dazwischen:

- client-eth0 und r1-eth0 in 10.0.1.0/24, Maske 255.255.255.0
- r1-eth1 und r2-eth0 in 10.0.12.0/30, Maske 255.255.255.252
- r2-eth1 und server-eth0 in 10.0.2.0/24, Maske 255.255.255.0

Getrennt sind die drei Netze an den Routern. 10.0.1.0/24 und 10.0.12.0/30 treffen sich nur in r1, an zwei verschiedenen Interfaces. 10.0.12.0/30 und 10.0.2.0/24 treffen sich nur in r2. Zwischen 10.0.1.0/24 und 10.0.2.0/24 liegen beide Router. `lo` gehört nicht dazu: jeder Namespace hat sein eigenes Loopback, die Adresse 127.0.0.1 ist nirgends ein gemeinsames Netz.

Erklärung

Ein gemeinsames Präfix in der Adresse reicht nicht. 10.0.1.2 und 10.0.2.2 beginnen beide mit 10, liegen aber in verschiedenen /24-Netzen. Es zählt die Maske, die `ip addr` hinter dem Schrägstrich nennt. /30 auf der Strecke zwischen r1 und r2 hat genau zwei nutzbare Adressen, .1 und .2. .0 ist die Netzadresse, .3 der Broadcast. Deshalb stehen in der Vorlage genau diese beiden Routeradressen.

Drei Netze: 10.0.1.0/24, 10.0.12.0/30, 10.0.2.0/24.

Aufgabe 6.2

Routingtabellen mit `ip route` untersuchen. Erklären, wie Daten von 10.0.1.2 nach 10.0.2.2 geleitet werden. Kleine Forwarding-Tabelle für r1 und r2 mit Next Hop.

Auf dem Client lag diese Tabelle:

```
default via 10.0.1.1 dev client-eth0
10.0.1.0/24 dev client-eth0 proto kernel scope link src 10.0.1.2
```

`ip route get 10.0.2.2` auf dem Client lieferte `10.0.2.2 via 10.0.1.1 dev client-eth0 src 10.0.1.2`. 10.0.2.2 steckt nicht in 10.0.1.0/24, also greift die Default-Route. Der nächste Hop ist r1.

Auf r1:

```
10.0.1.0/24 dev r1-eth0 proto kernel scope link src 10.0.1.1
10.0.2.0/24 via 10.0.12.2 dev r1-eth1
10.0.12.0/30 dev r1-eth1 proto kernel scope link src 10.0.12.1
```

`ip route get 10.0.2.2` auf r1 lieferte `via 10.0.12.2 dev r1-eth1 src 10.0.12.1`. Die längere Präfixübereinstimmung ist 10.0.2.0/24, nicht die Default-Route, denn r1 hat keine Default-Route.

Auf r2 lieferte `ip route get 10.0.2.2` die Zeile `10.0.2.2 dev r2-eth1 src 10.0.2.1`. Das Ziel liegt im angeschlossenen Netz, ein weiterer Hop ist nicht nötig. Der Server nimmt das Paket an 10.0.2.2 an.

Der Hinweg ist also client, dann r1, dann r2, dann server. Ein Ping dorthin kam mit TTL 62 zurück, nicht mit 64. Jeder der beiden Router zieht 1 ab. Die Rückrichtung nutzt auf r2 den Eintrag `10.0.1.0/24 via 10.0.12.1 dev r2-eth0`. Ohne den käme die Antwort nicht zurück, auch wenn der Hinweg stimmt.

Forwarding-Tabelle r1:

Ziel	Next Hop	Interface
10.0.1.0/24	direkt verbunden	r1-eth0
10.0.12.0/30	direkt verbunden	r1-eth1
10.0.2.0/24	10.0.12.2	r1-eth1

Forwarding-Tabelle r2:

Ziel	Next Hop	Interface
10.0.2.0/24	direkt verbunden	r2-eth1
10.0.12.0/30	direkt verbunden	r2-eth0
10.0.1.0/24	10.0.12.1	r2-eth0

Bei einer verbundenen Route ist der Next Hop das eigene Interface, nicht eine fremde Adresse. `ip_forward` stand auf r1 und r2 auf 1, auf client und server auf 0. Nur die Router leiten Pakete weiter, die nicht an sie selbst gerichtet sind.

Erklärung

Linux legt die verbundenen Routen selbst an, sobald `ip addr add` eine Adresse setzt. Die beiden Zeilen mit `via` kommen extra aus `start_lab.py`. Eine Default-Route auf dem Client ist nötig, weil 10.0.2.2 außerhalb von 10.0.1.0/24 liegt. r1 braucht keine Default-Route: er kennt das Servernetz explizit.

$$10.0.1.2 \rightarrow 10.0.1.1 \rightarrow 10.0.12.2 \rightarrow 10.0.2.2$$

Aufgabe 6.3

Auf r1 und r2 die bereitgestellten Routen entfernen und danach manuell neu setzen. Jeden Schritt dokumentieren. Danach `ping -c 3 10.0.2.2` vom Client zum Server.

Vor dem Löschen stand auf r1 die Zeile `10.0.2.0/24 via 10.0.12.2 dev r1-eth1`, auf r2 die Zeile `10.0.1.0/24 via 10.0.12.1 dev r2-eth0`. Das sind die bereitgestellten Routen. Die verbundenen Kernel-Routen bleiben, die hat niemand von Hand gesetzt.

Auf r1:

```
ip route del 10.0.2.0/24
```

Auf r2:

```
ip route del 10.0.1.0/24
```

Danach fehlten genau diese beiden Zeilen. r1 kannte 10.0.2.0/24 nicht mehr, r2 kannte 10.0.1.0/24 nicht mehr. Der Ping vom Client:

```
ping -c 2 -W 1 10.0.2.2
From 10.0.1.1 icmp_seq=1 Destination Net Unreachable
From 10.0.1.1 icmp_seq=2 Destination Net Unreachable
2 packets transmitted, 0 received, +2 errors, 100% packet loss
```

10.0.1.1 ist r1. Der Client schickt weiter an seinen Default-Hop, r1 findet kein Netz für 10.0.2.2 und antwortet mit ICMP Destination Net Unreachable. Die Leitung selbst ist nicht kaputt, die Route fehlt.

Neu setzen, so wie die Vorlage es vorher hatte:

```
# auf r1
ip route add 10.0.2.0/24 via 10.0.12.2
# auf r2
ip route add 10.0.1.0/24 via 10.0.12.1
```

Linux hat das Gerät selbst ergänzt, `dev r1-eth1` bzw. `dev r2-eth0`, weil der Next Hop in dem angeschlossenen /30 liegt. Der anschließende Ping:

```
ping -c 3 -W 2 10.0.2.2
64 bytes from 10.0.2.2: icmp_seq=1 ttl=62 time=101 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=62 time=60.3 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=62 time=60.6 ms
3 packets transmitted, 3 received, 0% packet loss, time 2003ms
rtt min/avg/max/mdev = 60.328/73.894/100.742/18.984 ms
```

Die späteren Pakete liegen bei 60 ms. Drei Links, hin und zurück, passen zu diesem Wert: die Vorlage setzt 10 ms pro Link, der ruhige Hin- und Rückweg braucht etwa 20 ms pro Link. Das erste Paket brauchte 101 ms, das ist die ARP-Auflösung, deshalb liegt der Mittelwert bei 73,894 ms und nicht bei 60. TTL 62 heißt: zwei Router, jeder zieht 1 von 64 ab.

Erklärung

Beide Richtungen müssen eine Route haben. Nur die Route auf r1 repariert den Hinweg. Die Antwort des Servers läuft über r2, und r2 braucht 10.0.1.0/24 via 10.0.12.1. Wer nur eine Seite setzt, sieht weiter Paketverlust, nur diesmal still, ohne die ICMP-Meldung von r1.

Nach dem Neueintrag: 3/3, 0 % Verlust, TTL 62.

Aufgabe 6.4

Subnetze planen für 200 Endgeräte, 50 Server, 20 Management-Geräte und eine Punkt-zu-Punkt-Verbindung zwischen zwei Routern. Je Netzadresse, Präfix bzw. Maske, erste und letzte nutzbare Hostadresse, Broadcast, Anzahl möglicher Hosts. Mit der RN-Lab-Topologie vergleichen und die Präfixe begründen.

Die Angabe nennt keinen Adressblock. Ich nehme den privaten Block 10.10.0.0/23 und vergebe die Netze von groß nach klein, ohne Überlappung. Nutzbare Hosts sind $2^{32-p} - 2$, Netz- und Broadcastadresse zählen nicht mit.

Endgeräte, Bedarf 200. Acht Hostbits reichen, sieben nicht:

$2^8 - 2 = 254$	/24, acht Hostbits
$254 \geq 200$	Vergleich mit dem Bedarf
$2^7 - 2 = 126$	/25, sieben Hostbits
$126 < 200$	/25 ist zu klein

Also 10.10.0.0/24, Maske 255.255.255.0. Erste nutzbare Adresse 10.10.0.1, letzte 10.10.0.254, Broadcast 10.10.0.255, 254 mögliche Hosts.

Server, Bedarf 50:

$2^6 - 2 = 62$	/26
$62 \geq 50$	Vergleich mit dem Bedarf
$2^5 - 2 = 30$	/27
$30 < 50$	/27 ist zu klein

Der nächste freie Block nach 10.10.0.0/24 ist 10.10.1.0/26. Maske 255.255.255.192. Erste Adresse 10.10.1.1, letzte 10.10.1.62, Broadcast 10.10.1.63, 62 Hosts.

Management, Bedarf 20. Der /26-Block endet bei 10.10.1.63, der nächste ausgerichtete /27 beginnt bei 10.10.1.64:

$2^5 - 2 = 30$	/27
$30 \geq 20$	Vergleich mit dem Bedarf
$2^4 - 2 = 14$	/28
$14 < 20$	/28 ist zu klein

10.10.1.64/27, Maske 255.255.255.224. Erste Adresse 10.10.1.65, letzte 10.10.1.94, Broadcast 10.10.1.95, 30 Hosts.

Punkt-zu-Punkt, Bedarf 2:

$2^2 - 2 = 2$	/30
$2 \geq 2$	genau die zwei Routerinterfaces
$2^1 - 2 = 0$	/31 unter der klassischen Zählung
$0 < 2$	kein Netz- und Broadcastadresse übrig

Nach 10.10.1.95 beginnt der nächste /30-Block bei 10.10.1.96. Maske 255.255.255.252. Nutzbar sind 10.10.1.97 und 10.10.1.98, Broadcast 10.10.1.99, 2 Hosts.

Die vier Netze überlappen nicht. 10.10.0.0/24 und 10.10.1.0/26 liegen in verschiedenen dritten Oktetten. 10.10.1.64 liegt hinter 10.10.1.63, und 10.10.1.96 liegt hinter 10.10.1.95.

Im Labor ist das Client-LAN 10.0.1.0/24 und das Server-LAN 10.0.2.0/24, obwohl dort je nur zwei Interfaces hängen. /24 ist dort Bequemlichkeit, nicht der Bedarf. Mein Endgeräte-Netz ist auch /24, aber weil 200 Hosts in ein /25 nicht passen. Für 50 Server reicht /26, ein /24 würde 192 Adressen verschenken. Die Laborstrecke zwischen den Routern ist 10.0.12.0/30 mit 10.0.12.1 und 10.0.12.2. Dieselbe Länge nutze ich für die Punkt-zu-Punkt-Verbindung: zwei Interfaces, zwei nutzbare Adressen. /31 wäre nach RFC 3021 auf einer reinen Punkt-zu-Punkt-Strecke möglich, hätte aber keine eigene Broadcastadresse. Die Aufgabe will die Broadcastadresse in der Tabelle, und das Labor verwendet /30. Deshalb /30.

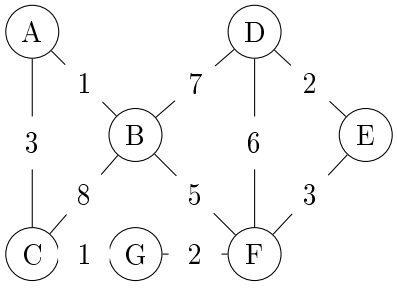
Erklärung

VLSM heißt nur: die Präfixe dürfen verschieden lang sein, die Blöcke müssen an ihrer eigenen Größe ausgerichtet sein. Ein /27 darf nicht bei 10.10.1.50 anfangen. 64 ist durch 32 teilbar, 96 durch 4. Deshalb die Sprünge 0, 64 und 96 im dritten Netz. Wer überall /24 nimmt, erfüllt den Hostbedarf auch, verbraucht aber viermal so viel Adressraum für Server und Management.

10.10.0.0/24, 10.10.1.0/26, 10.10.1.64/27, 10.10.1.96/30

Aufgabe 6.5

Kürzeste Wege von A zu allen Knoten mit Dijkstra, Tabelle wie in der Vorlesung (Kurose/Ross). Kanten aus Abbildung 1, ungerichtet: A–B 1, A–C 3, B–C 8, B–D 7, B–F 5, C–G 1, D–E 2, D–F 6, E–F 3, G–F 2.



N' enthält die Knoten, deren Distanz fest ist. $D(v), p(v)$ ist die vorläufige Distanz und der Vorgänger. Aktualisiert wird nur, wenn der neue Weg strikt kürzer ist. Bei Gleichstand bleibt der erste Vorgänger. Alle Kantenkosten sind positiv, deshalb ist die Distanz eines Knotens endgültig, sobald er in N' kommt.

Start: $D(A) = 0$, alle anderen ∞. A ist der einzige Kandidat.

A wird fest, $N' = \{A\}$. Die Kanten von A:

$D(B) = 0 + 1$	Kante A–B
$D(B) = 1$	$0 + 1 = 1$, vorher ∞
$D(C) = 0 + 3$	Kante A–C
$D(C) = 3$	$0 + 3 = 3$, vorher ∞

Vorgänger von B und C ist A.

Unter den übrigen Knoten hat B die kleinste Distanz, $D(B) = 1$. $N' = \{A, B\}$.

$1 + 7 = 8$	B–D, $8 < \infty$, $p(D) = B$
$1 + 8 = 9$	B–C, $9 > 3$, C bleibt
$1 + 5 = 6$	B–F, $6 < \infty$, $p(F) = B$

Als Nächstes C mit $D(C) = 3$. $N' = \{A, B, C\}$.

$3 + 1 = 4$	C–G, $4 < \infty$, $p(G) = C$

Dann G mit $D(G) = 4$. $N' = \{A, B, C, G\}$.

$4 + 2 = 6$	G–F, $6 = 6$, kein strikt kleinerer Weg

F bleibt bei 6 über B.

Dann F mit $D(F) = 6$. $N' = \{A, B, C, G, F\}$.

$6 + 6 = 12$	F–D, $12 > 8$, D bleibt
$6 + 3 = 9$	F–E, $9 < \infty$, $p(E) = F$

Dann D mit $D(D) = 8$. $N' = \{A, B, C, G, F, D\}$.

$8 + 2 = 10$	D–E, $10 > 9$, E bleibt

Zuletzt E mit $D(E) = 9$. Keine offene Kante mehr.

Schritt	N'	$D(B), p(B)$	$D(C), p(C)$	$D(D), p(D)$	$D(E), p(E)$	$D(F), p(F)$	$D(G), p(G)$
0, A fest	A	1, A	3, A	∞	∞	∞	∞
1, B	A,B	1, A	3, A	8, B	∞	6, B	∞
2, C	A,B,C	1, A	3, A	8, B	∞	6, B	4, C
3, G	A,B,C,G	1, A	3, A	8, B	∞	6, B	4, C
4, F	A,B,C,G,F	1, A	3, A	8, B	9, F	6, B	4, C
5, D	A,B,C,D,F,G	1, A	3, A	8, B	9, F	6, B	4, C
6, E	alle	1, A	3, A	8, B	9, F	6, B	4, C

Wege im Vorgängerbaum, und der zweite kürzeste Weg bei Gleichstand:

Ziel	Kosten	Weg
B	1	A–B
C	3	A–C
G	4	A–C–G
F	6	A–B–F, ebenso A–C–G–F
D	8	A–B–D
E	9	A–B–F–E, ebenso A–C–G–F–E

Die zweite Variante nach F kostet 3+1+2 = 6, gleich viel wie 1+5. Nach E kosten beide Fortsetzungen über F noch einmal 3, also 9. Der Baum behält den ersten Vorgänger, die Tabelle nennt beide Wege.

Probe für D über F: $6 + 6 = 12$, größer als 8. Probe für E über D: $8 + 2 = 10$, größer als 9.

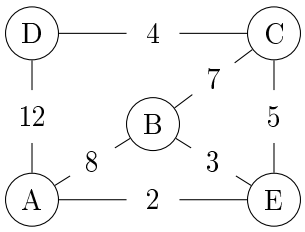
Erklärung

Dijkstra nimmt immer den noch nicht festen Knoten mit der kleinsten Distanz. Deshalb kommt G vor F, obwohl F schon einen Wert hat: 4 ist kleiner als 6. Bei Gleichstand nicht umschreiben, sonst hängt der Vorgänger von der Kantenreihenfolge ab und zwei korrekte Läufe sehen verschieden aus. Die Kosten bleiben trotzdem gleich.

$$\boxed{B{:}\,1,\ C{:}\,3,\ G{:}\,4,\ F{:}\,6,\ D{:}\,8,\ E{:}\,9}$$

Aufgabe 6.6

Distance-Vector für Knoten B, Abbildung 2. Kanten ungerichtet: A–B 8, A–D 12, A–E 2, B–C 7, B–E 3, C–D 4, C–E 5. Initiale Distanztabelle von B und die Tabelle nach jedem Schritt. Der Algorithmus beginnt mit der billigsten Kante. Änderungen markieren, den empfangenen Distanzvektor jedes Schritts angeben.



Die Angabe legt nicht fest, ob nur B seine Nachbarn der Reihe nach abfragt oder das ganze Netz Kanten nach Kosten abarbeitet. Ich nehme die strengere Lesart: in jeder Runde wird jede gerichtete Kante einmal zugestellt, billigste zuerst. Die billigste Kante im Netz ist A–E mit Kosten 2. Bei B kommen die Nachrichten dadurch in der Reihenfolge E (3), C (7), A (8) an. Der Sender schickt den Vektor, den er zum Zustellzeitpunkt hat, Reihenfolge (A, B, C, D, E). Eine Zelle $T_B(z, v)$ ist $c(B, v) + D_v(z)$. Ein Stern markiert eine Zelle, die sich gegenüber der vorigen Tabelle von B geändert hat.

Initiale Tabelle. B kennt nur seine eigenen Kanten, noch keine fremden Vektoren. Eigene Distanz $D_B(B) = 0$, die Zeile B steht deshalb nicht in der Nachbartabelle.

Ziel	über E (3)	über C (7)	über A (8)	Minimum
A	∞	∞	8	8
C	∞	7	∞	7
D	∞	∞	∞	∞
E	3	∞	∞	3

Distanzvektor von B: $(8, 0, 7, \infty, 3)$.

Schritt 1, Nachricht von E. Vorher ist A→E zugestellt worden, Kosten 2. E setzt damit $D_E(D) = 2 + 12 = 14$ und schickt $(2, 3, 5, 14, 0)$.

$$\begin{aligned} 3 + 2 &= 5 && \text{A über E, vorher } \infty \\ 3 + 5 &= 8 && \text{C über E} \\ 3 + 14 &= 17 && \text{D über E} \\ 3 + 0 &= 3 && \text{E über E, unverändert} \end{aligned}$$

Ziel	über E (3)	über C (7)	über A (8)	Minimum
A	5^*	∞	8	5^*
C	8^*	7	∞	7
D	17^*	∞	∞	17^*
E	3	∞	∞	3

Neuer Vektor von B: $(5, 0, 7, 17, 3)$. 17 ist nur ein Zwischenwert. E kennt D bislang nur über A.

Schritt 2, Nachricht von C. C hat inzwischen von E gelernt und schickt $(7, 7, 0, 4, 5)$. Die 7 nach A ist $c(C, E) + D_E(A) = 5 + 2$. Die 4 nach D ist die direkte Kante C–D.

$7 + 7 = 14$	A über C
$7 + 0 = 7$	C über C, unverändert
$7 + 4 = 11$	D über C, besser als 17
$7 + 5 = 12$	E über C

Ziel	über E (3)	über C (7)	über A (8)	Minimum
A	5	14^*	8	5
C	8	7	∞	7
D	17	11^*	∞	11^*
E	3	12^*	∞	3

Vektor von B: $(5, 0, 7, 11, 3)$.

Schritt 3, Nachricht von A. A hat von E schon $D_A(B) = 2 + 3 = 5$ und $D_A(C) = 2 + 5 = 7$. Empfangen: $(0, 5, 7, 12, 2)$.

$8 + 0 = 8$	A über A, gleich dem Startwert
$8 + 7 = 15$	C über A
$8 + 12 = 20$	D über A
$8 + 2 = 10$	E über A

Ziel	über E (3)	über C (7)	über A (8)	Minimum
A	5	14	8	5
C	8	7	15^*	7
D	17	11	20^*	11
E	3	12	10^*	3

Das Minimum ändert sich nicht. Vektor von B bleibt $(5, 0, 7, 11, 3)$.

In Runde 2 schickt E $(2, 3, 5, 9, 0)$, weil E die Kante C–D inzwischen günstiger findet als den Umweg über A: $c(E, C) + 4 = 5 + 4 = 9$.

$3 + 9 = 12$	D über E, 17 wird 12, immer noch schlechter als 11

Die übrigen Minima bleiben. C schickt wieder $(7, 7, 0, 4, 5)$, keine neue Zelle. A schickt $(0, 5, 7, 11, 2)$, damit wird D über A zu $8 + 11 = 19$, immer noch schlechter als 11. Runde 3 ändert keinen Vektor im Netz mehr.

Würde B die initialen Nachbarvektoren nehmen, bevor A→E gelaufen ist, bliebe D über E in Schritt 1 bei ∞ statt 17. Das Minimum nach der Nachricht von C wäre trotzdem 11. Die Endwerte hängen nicht von dieser Reihenfolge ab.

Kürzeste Wege von B zur Probe, unabhängig mit Dijkstra auf demselben Graphen: A über E mit $3 + 2 = 5$, C direkt mit 7, D über C mit $7 + 4 = 11$, E direkt mit 3. Dieselben Zahlen.

Erklärung

Die Spalte ist nicht der Distanzvektor des Nachbarn, sondern Kosten bis zum Nachbarn plus dessen Distanz zum Ziel. Deshalb kann in der Spalte ein Umweg stehen, den B nie wählt. 17 nach D über E ist so ein Umweg: E hat D nur über A gemeldet, und A hängt mit 12 an D. Sobald C seine direkte 4 meldet, fällt das Minimum auf 11. Split Horizon ist hier nicht dabei. Die Aufgabe verlangt den reinen Distance-Vector.

$$D_B = (A{:}\,5,\ B{:}\,0,\ C{:}\,7,\ D{:}\,11,\ E{:}\,3)$$

Aufgabe 6.7

In `topology.py` einen weiteren Client anlegen, eine passende IP vergeben, ihn mit r1 verbinden, eine Default-Route setzen und die Verbindung zum Server testen. `tcp_client.py` und `tcp_server.py` nicht ändern. Die Änderungen dokumentieren und erklären, warum die beiden Programme bleiben können.

Der Anhang nennt `topology.py` als unveränderlich, die Aufgabe 6.7 sagt ab dort ausdrücklich, die Datei zu bearbeiten. Ich folge 6.7. `start_lab.py` bleibt wie im Anhang unverändert. Die Messungen von 6.1 bis 6.3 gelten für die Vorlage davor.

Der neue Client soll laut Skizze an die linke Strecke zwischen client und r1. Das ist dasselbe IP-Netz, kein neues. Eine zweite Punkt-zu-Punkt-Leitung an r1 mit einer zweiten Adresse aus 10.0.1.0/24 wäre kein gemeinsames LAN: ARP bliebe an dem einen Interface hängen. Deshalb ersetzt eine lernende Bridge `s1` (`OVSBridge`, `failMode=standalone`) die direkte Leitung. client-eth0, client2-eth0 und r1-eth0 hängen an s1. s1 hat keine IP und ist kein zusätzlicher IP-Hop.

client2 bekommt 10.0.1.3/24. .0 ist die Netzadresse, .1 ist r1, .2 ist der alte Client, .255 der Broadcast. Die Default-Route ist `via 10.0.1.1`. Die setzt Mininet beim Start des Hosts. `configure_base_network` in der unveränderten `start_lab.py` fasst client2 nicht an, die Adresse bleibt also stehen. r2 hat schon `10.0.1.0/24 via 10.0.12.1`. Diese Route gilt für .2 und für .3. Eine Extra-Route ist nicht nötig.

Nach dem Start, immer noch über die unveränderte Konfigurationsfunktion, zeigte client2:

```
client2-eth0 10.0.1.3/24
default via 10.0.1.1 dev client2-eth0
10.0.1.0/24 dev client2-eth0 proto kernel scope link src 10.0.1.3
```

Ping von client2 nach 10.0.2.2, gemessen am 30.10.2026:

```
64 bytes from 10.0.2.2: icmp_seq=1 ttl=62 time=162 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=62 time=80.1 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=62 time=80.2 ms
3 packets transmitted, 3 received, 0% packet loss, time 2002ms
rtt min/avg/max/mdev = 80.126/107.291/161.567/38.378 ms
```

Die späteren Pakete liegen bei 80 ms. Vier Linkstücke (client2–s1, s1–r1, r1–r2, r2–server), hin und zurück, passen dazu. Der Mittelwert 107,291 ms ist höher, weil das erste Paket 162 ms brauchte. Das ist wieder ARP. TTL 62 heißt zwei Router. Die Bridge ist kein IP-Hop und zieht nichts ab. Ping vom Server nach 10.0.1.3: 2 gesendet, 2 empfangen, TTL 62, rtt min/avg/max/mdev = 80,132/80,214/80,297/0,082 ms. Der alte Client erreicht den Server weiter, 0 % Verlust, TTL 62. Sein ruhiges Paket lag bei 80,6 ms, nicht mehr bei 60 ms, weil auch er jetzt über die Bridge geht.

Die TCP-Programme der Vorlage liefen unverändert, Server auf dem Server-Host, Client auf client2:

```
Connected to 10.0.2.2:5000
```

```
Reply: b'Network layer test 1\n'
Reply: b'Network layer test 2\n'
Reply: b'Network layer test 3\n'
```

Der Client bindet keine Quelladresse. Der Kernel nimmt 10.0.1.3 und einen freien Quellport. Der Server lauscht auf 10.0.2.2:5000 und antwortet an die Adresse, die `accept` liefert. Welcher Host im Netz 10.0.1.0/24 die Verbindung öffnet, steht nicht im Programm. Routing und die Bridge tragen die Segmente. Deshalb dürfen die beiden Dateien bleiben, wie sie sind.

Erklärung

Eine neue Adresse im Servernetz, etwa 10.0.2.3, wäre die falsche Seite: die Skizze hängt den neuen Client links an r1. 10.0.1.3 ist frei und liegt im selben Netz wie das Gateway 10.0.1.1. Ohne die Default-Route würde client2 10.0.2.2 nicht kennen, das verbundene Netz ist nur 10.0.1.0/24. Die Bridge ist der Punkt, an dem man sich leicht vertut: zwei Interfaces von r1 im selben /24 sind kein Ersatz für eine gemeinsame Schicht-2-Verbindung.

client2 = 10.0.1.3/24 via 10.0.1.1, Ping und TCP zum Server ok